

UNIT-I

Object Oriented System Development: Understanding Object Oriented Development, Understanding Object Concepts, Benefits of Object Oriented Development.

Java Programming Fundamentals: Introduction, Overview of Java, Data Type, Variables and Arrays, Operators, Control statements, Classes, Methods, Inheritance, Packages and Interfaces, Inner Classes.

1.1 UNDERSTANDING JAVA'S OBJECT-ORIENTED DEVELOPMENT

Object Oriented Programming is a programming concept that works on the principle that objects are the most important part of your program. It allows users create the objects that they want and then create methods to handle those objects. Manipulating these objects to get results is the goal of Object Oriented Programming. Object Oriented Programming popularly known as OOP, is used in a modern programming language like Java.

Java is object-oriented. Object-oriented languages are better than “Do this/Do that” languages because they organize data in a way that lets people do all kinds of things with it. To modify the data, you can build on what you already have, rather than scrap everything you’ve done and start over each time you need to do something new. Although computer programmers are generally smart people, they took awhile to figure this out.

OBJECTS AND THEIR CLASSES

In an object-oriented language, you use objects *and* classes to organize your data.

Imagine that you’re writing a computer program to keep track of the houses in a new condominium development. The houses differ only slightly from one another. Each house has a distinctive siding color, an indoor paint color, a kitchen cabinet style, and so on. In your object-oriented computer program, each house is an object.

But objects aren’t the whole story. Although the houses differ slightly from one another, all the houses share the same list of characteristics. For instance, each house has a characteristic known as *siding color*. Each house has another characteristic known as *kitchen cabinet style*. In your object-oriented program, you need a master list containing all the characteristics that a house object can possess. This master list of characteristics is called a *class*.

So there you have it. Object-oriented programming is misnamed. It should really be called “programming with classes and objects.”

Notice that the word *classes* were listed first? Think again about a housing development that's under construction. Somewhere on the lot, in a rickety trailer parked on bare dirt, is a master list of characteristics known as a blueprint. An architect's blueprint is like an object-oriented programmer's class. A blueprint is a list of characteristics that each house will have. The blueprint says, "siding." The actual house object has gray siding. The blueprint says, "kitchen cabinet." The actual house object has Louis XIV kitchen cabinets.

A year after you create the blueprint, you use it to build ten houses. It's the same with classes and objects. First, the programmer writes code to describe a class. Then when the program runs, the computer creates objects from the class.

Core OOPS concepts are

1) Class

The class is a group of similar entities. It is only a logical component and not the physical entity. For example, if you had a class called "Expensive Cars" it could have objects like Mercedes, BMW, Toyota, etc. Its properties (data) can be price or speed of these cars. While the methods may be performed with these cars are driving, reverse, braking etc.

2) Object

An object can be defined as an instance of a class, and there can be multiple instances of a class in a program. An Object contains both the data and the function, which operates on the data. For example - chair, bike, marker, pen, table, car, etc.

3) Inheritance

Inheritance is an OOPS concept in which one object acquires the properties and behaviors of the parent object. It's creating a parent-child relationship between two classes. It offers robust and natural mechanism for organizing and structure of any software.

4) Polymorphism

Polymorphism refers to the ability of a variable, object or function to take on multiple forms. For example, in English, the verb "run" has a different meaning if you use it with "a laptop," "a foot race, and "business.&rdquo Here, we understand the meaning of "run" based on the other words used along with it. The same also applied to Polymorphism.

5) Abstraction

An abstraction is an act of representing essential features without including background details. It is a technique of creating a new data type that is suited for a specific application. For example, while driving a car, you do not have to be concerned with its internal working. Here you just need to concern about parts like steering wheel, Gears, accelerator, etc.

6) Encapsulation

Encapsulation is an OOP technique of wrapping the data and code. In this OOPS concept, the variables of a class are always hidden from other classes. It can only be accessed using the methods of their current class. For example - in school, a student cannot exist without a class.

7) Association

Association is a relationship between two objects. It defines the diversity between objects. In this OOP concept, all object have their separate lifecycle, and there is no owner. For example, many students can associate with one teacher while one student can also associate with multiple teachers.

8) Aggregation

In this technique, all objects have their separate lifecycle. However, there is ownership such that child object can't belong to another parent object. For example consider class/objects department and teacher. Here, a single teacher can't belong to multiple departments, but even if we delete the department, the teacher object will never be destroyed.

9) Composition

A composition is a specialized form of Aggregation. It is also called "death" relationship. Child objects do not have their lifecycle so when parent object deletes all child object will also delete automatically. For that, let's take an example of House and rooms. Any house can have several rooms. One room can't become part of two different houses. So, if you delete the house room will also be deleted.

1.2 Advantages of OOPS:

1. OOP offers easy to understand and a clear modular structure for programs.
2. Objects created for Object-Oriented Programs can be reused in other programs. Thus it saves significant development cost.
3. Large programs are difficult to write, but if the development and designing team follow OOPS concept then they can better design with minimum flaws.
4. It also enhances program modularity because every object exists independently.

1.3 Data types in Java

There are majorly two types of languages. First one is **Statically typed language** where each variable and expression type is already known at compile time. Once a variable is declared to be of a certain data type, it cannot hold values of other data types. Example: C, C++, Java. Other, **Dynamically typed languages**: These languages (Ruby, Python) can receive different data types over the time.

Java is **statically typed and also a strongly typed language** because in Java, each type of data (such as integer, character, hexadecimal, packed decimal, and so forth) is predefined as part of the programming language and all constants or variables defined for a given program must be described with one of the data types.

Java has two categories of data:

- Primitive data (e.g., number, character)
- Object data (programmer created types)

TYPE	DESCRIPTION	DEFAULT	SIZE	EXAMPLE LITERALS
boolean	true or false	false	1 bit	true, false
byte	twos complement integer	0	8 bits	(none)
char	unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\', '\n', '\b'
short	twos complement integer	0	16 bits	(none)
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d

boolean: boolean data type represents only one bit of information **either true or false** . Values of type boolean are not converted implicitly or explicitly (with casts) to any other type. But the programmer can easily write conversion code.

```
// A Java program to demonstrate boolean data type
class GeeksforGeeks
{
    public static void main(String args[])
    {
        boolean b = true;
```

```

        if (b == true)
            System.out.println("Hi Geek");
    }
}

```

byte: The byte data type is an 8-bit signed two's complement integer. The byte data type is useful for saving memory in large arrays.

- **Size:** 8-bit
- **Value:** -128 to 127

// Java program to demonstrate byte data type in Java
class GeeksforGeeks

```

{
    public static void main(String args[])
    {
        byte a = 126;

        // byte is 8 bit value
        System.out.println(a);

        a++;
        System.out.println(a);

        // It overflows here because
        // byte can hold values from -128 to 127
        a++;
        System.out.println(a);

        // Looping back within the range
        a++;
        System.out.println(a);
    }
}

```

short: The short data type is a 16-bit signed two's complement integer. Similar to byte, use a short to save memory in large arrays, in situations where the memory savings actually matters.

- **Size:** 16 bit
- **Value:** -32,768 to 32,767 (inclusive)

int

It is a 32-bit signed two's complement integer.

- **Size:** 32 bit
- **Value:** -2^{31} to $2^{31}-1$

Note: In Java SE 8 and later, we can use the int data type to represent an unsigned 32-bit integer, which has value in range $[0, 2^{32}-1]$. Use the Integer class to use int data type as an unsigned integer.

long: The long data type is a 64-bit two's complement integer.

- Size: 64 bit
- Value: -2^{63} to $2^{63}-1$.

Note: In Java SE 8 and later, you can use the long data type to represent an unsigned 64-bit long, which has a minimum value of 0 and a maximum value of $2^{64}-1$. The Long class also contains methods like compareUnsigned, divideUnsigned etc to support arithmetic operations for unsigned long.

Floating point Numbers : float and double

float: The float data type is a single-precision 32-bit IEEE 754 floating point. Use a float (instead of double) if you need to save memory in large arrays of floating point numbers.

- **Size:** 32 bits
- **Suffix :** F/f Example: 9.8f

double: The double data type is a double-precision 64-bit IEEE 754 floating point. For decimal values, this data type is generally the default choice.

Note: Both float and double data types were designed especially for scientific calculations, where approximation errors are acceptable. If accuracy is the most prior concern then, it is recommended not to use these data types and use BigDecimal class instead.

Please see this for details: *Rounding off errors in Java*

char

The char data type is a single 16-bit Unicode character. A char is a single character.

- Value: '\u0000' (or 0) to '\uffff' 65535

// Java program to demonstrate primitive data types in Java
class GeeksforGeeks

```
{  
    public static void main(String args[])  
    {  
        // declaring character  
        char a = 'G';  
  
        // Integer data type is generally  
        // used for numeric values  
        int i=89;  
  
        // use byte and short if memory is a constraint  
        byte b = 4;  
  
        // this will give error as number is  
        // larger than byte range
```

```

// byte b1 = 7888888955;

short s = 56;

// this will give error as number is
// larger than short range
// short s1 = 87878787878;

// by default fraction value is double in java
double d = 4.355453532;

// for float use 'f' as suffix
float f = 4.7333434f;

System.out.println("char: " + a);
System.out.println("integer: " + i);
System.out.println("byte: " + b);
System.out.println("short: " + s);
System.out.println("float: " + f);
System.out.println("double: " + d);
}
}

```

1.4 Java Array

Normally, an array is a collection of similar type of elements that have a contiguous memory location.

Java array is an object which contains elements of a similar data type. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.

Array in java is index-based, the first element of the array is stored at the 0 index.

Advantages

- **Code Optimization:** It makes the code optimized, we can retrieve or sort the data efficiently.
- **Random access:** We can get any data located at an index position.

Disadvantages

- **Size Limit:** We can store only the fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in Java which grows automatically.

There are two types of array.

- Single Dimensional Array
- Multidimensional Array

1. EXAMPLE ARRAY INITIALIZATION (single dimension)

```
class Testarray{
public static void main(String args[]){
int a[]=new int[5];//declaration and instantiation
a[0]=10;//initialization
a[1]=20;
a[2]=70;
a[3]=40;
a[4]=50;
//traversing array
for(int i=0;i<a.length;i++)//length is the property of array
System.out.println(a[i]);
}}
```

2. EXAMPLE ARRAY INITIALIZATION (single dimension)

```
class Testarray1{
public static void main(String args[]){
int a[]={33,3,4,5};//declaration, instantiation and initialization
//printing array
for(int i=0;i<a.length;i++)//length is the property of array
System.out.println(a[i]);
}}
```

Passing Array to Method in Java

```
class Testarray2{
//creating a method which receives an array as a parameter
static void min(int arr[]){
int min=arr[0];
for(int i=1;i<arr.length;i++)
if(min>arr[i])
min=arr[i];
System.out.println(min);
}
public static void main(String args[]){
int a[]={33,3,4,5};//declaring and initializing an array
min(a);//passing array to method
}}
```

Multidimensional Array in Java

```
class Testarray3{
public static void main(String args[]){
//declaring and initializing 2D array
int arr[][]={{1,2,3},{2,4,5},{4,4,5}};
//printing 2D array
for(int i=0;i<3;i++){
for(int j=0;j<3;j++){
System.out.print(arr[i][j]+" ");
}
System.out.println();
}
}}
```

1.5 Operators in java

Operator in java is a symbol that is used to perform operations. For example: +, -, *, / etc.

There are many types of operators in java which are given below:

- Unary Operator,
- Arithmetic Operator,
- Shift Operator,
- Relational Operator,
- Bitwise Operator,
- Logical Operator,
- Ternary Operator and
- Assignment Operator.

Operator Type	Category	Precedence
Unary	postfix	<i>expr++ expr--</i>
	prefix	<i>++expr --expr +expr -expr ~ !</i>
Arithmetic	multiplicative	<i>* / %</i>
	additive	<i>+ -</i>
Shift	shift	<i><< >> >>></i>
Relational	comparison	<i>< > <= >= instanceof</i>
	equality	<i>== !=</i>
Bitwise	bitwise AND	<i>&</i>

	bitwise exclusive OR	$\wedge$
	bitwise inclusive OR	$ $
Logical	logical AND	$\&\&$
	logical OR	$ $
Ternary	ternary	$? :$
Assignment	assignment	$= += -= *= /= \% = \&= \wedge= = \ll = \gg = \ggg =$

1.6 Control statements

A program executes from top to bottom except when we use control statements, we can control the order of execution of the program, based on logic and values.

In Java, control statements can be divided into the following three categories:

- **Selection Statements**
- **Iteration Statements**
- **Jump Statements**

Selection statements can be divided into the following categories:

1. The if and if-else statements
2. The if-else statements
3. The if-else-if statements
4. The switch statements

Example if-else demo

```
import java.util.Scanner;
public class IfElseDemo
{
    public static void main( String[] args )
    {
        int age;
        Scanner inputDevice = new Scanner( System.in );
        System.out.print( "Please enter Age: " );
        age = inputDevice.nextInt();
        if ( age >= 18 )
            System.out.println( "above 18 " );
        else
            System.out.println( "below 18" );
    }
}
```

Example switch

```
import java.util.Scanner;

public class SwitchDemo
```

```

{
    public static void main( String[] args )
    {
        int age;
        Scanner inputDevice = new Scanner( System.in );
        System.out.print( "Please enter Age: " );
        age = inputDevice.nextInt();
        switch ( age )
        {
            case 18:
                System.out.println( "age 18" );
                break;
            case 19:
                System.out.println( "age 19" );
                break;
            default:
                System.out.println( "not matched" );
                break;
        }
    }
}

```

Iteration Statements

Repeating the same code fragment several times until a specified condition is satisfied is called iteration. Iteration statements execute the same set of instructions until a termination condition is met.

Java provides the following loop for iteration statements:

- The while loop
- The for loop
- The do-while loop
- The for each loop

Example while loop

```

public class WhileDemo
{
    public static void main( String[] args )
    {
        int i = 0;
        while ( i < 5 )
        {
            System.out.println( "Value :: " + i );
            i++;
        }
    }
}

```

Example do while

```
public class DoWhileDemo
{
    public static void main( String[] args )
    {
        int i = 0;
        do
        {
            System.out.println( "value :: " + i );
            i++;
        }
        while ( i < 5);
    }
}
```

FOR loop example

```
public class UnLabeledBreakDemo
{
    public static void main( String[] args )
    {
        for ( int var = 0; var < 5; var++ )
        {
            System.out.println( "Var is : " + var );
            if ( var == 3 )
                break;
        }
    }
}
```

Jump Statements

Jump statements are used to unconditionally transfer the program control to another part of the program.

Java provides the following jump statements:

- break statement
- continue statement
- return statement

Break Statement

The break statement immediately quits the current iteration and goes to the first statement following the loop. Another form of break is used in the switch statement.

The break statement has the following two forms:

- Labeled Break Statement
- Unlabeled Break Statement

Labeled Break Statement: This is used for when we want to jump the program control out of nested loops or multiple loops.

```
public class LabeledBreakDemo
{
    public static void main( String[] args )
    {
```

```

    Outer: for ( int var1 = 0; var1 < 5; var1++ )
    {
        for ( int var2 = 1; var2 < 5; var2++ )
        {
            System.out.println( "var1:" + var1 + ", var2:" + var2 );
            if ( var1 == 3 )
                break Outer;
        }
    }
}

```

Unlabeled Break Statement: This is used to jump program control out of the specific loop on the specific condition.

```

public class UnLabeledBreakDemo
{
    public static void main( String[] args )
    {
        for ( int var = 0; var < 5; var++ )
        {
            System.out.println( "Var is : " + var );
            if ( var == 3 )
                break;
        }
    }
}

```

Continue Statement

The continue statement is used when you want to continue running the loop with the next iteration and want to skip the rest of the statements of the body for the current iteration.

The continue statement has the following two forms:

- Labeled Continue Statement- This statement skips the current iteration of the loop with the specified label.

```

public class LabeledContinueDemo
{
    public static void main( String[] args )
    {
        Outer: for ( int var1 = 0; var1 < 5; var1++ )
        {
            for ( int var2 = 0; var2 < 5; var2++ )
            {
                if ( var2 == 2 )
                    continue Outer;
                System.out.println( "var1:" + var1 + ", var2:" + var2 );
            }
        }
    }
}

```

- **Unlabeled Continue Statement-** This statement skips the current iteration of the innermost for, while and do-while loop.

```
public class UnlabeledContinueDemo
{
    public static void main( String[] args )
    {
        for ( int var1 = 0; var1 < 4; var1++ )
        {
            for ( int var2 = 0; var2 < 4; var2++ )
            {
                if ( var2 == 2 )
                    continue;
                System.out.println( "var1:" + var1 + ", var2:" + var2 );
            }
        }
    }
}
```

Return Statement

The return statement is used to immediately quit the current method and return to the calling method. It is mandatory to use a return statement for non-void methods to return a value.

```
public class ReturnDemo
{
    public static void main( String[] args )
    {
        ReturnDemo returnDemo = new ReturnDemo();
        System.out.println( "No : " + returnDemo.returnCall() );
    }

    int returnCall()
    {
        return 5;
    }
}
```

1.6 Classes in java

A class is a user defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. In general, class declarations can include these components, in order:

1. **Modifiers :** A class can be public or has default access (Refer this for details).
2. **Class name:** The name should begin with a initial letter (capitalized by convention).
3. **Superclass(if any):** The name of the class's parent (superclass), if any, preceded by the keyword extends. A class can only extend (subclass) one parent.
4. **Interfaces(if any):** A comma-separated list of interfaces implemented by the class, if any, preceded by the keyword implements. A class can implement more than one interface.
5. **Body:** The class body surrounded by braces, { }.

Example classes

```
class Lamp {
    boolean isOn;
    void turnOn() {
        isOn = true;
    }
    void turnOff() {
        isOn = false;
    }
}
class ClassObjectsExample {
    public static void main(String[] args) {
        Lamp l1 = new Lamp(); // create l1 object of Lamp class
        Lamp l2 = new Lamp(); // create l2 object of Lamp class
    }
}
```

1.7 Methods in Java

A method is a collection of statements that perform some specific task and return result to the caller. A method can perform some specific task without returning anything. Methods allow us to **reuse** the code without retyping the code. In Java, every method must be part of some class which is different from languages like C, C++ and Python.

Methods are **time savers** and help us to **reuse** the code without retyping the code.

Method Declaration

In general, method declarations has six components :

- **Modifier-**: Defines **access type** of the method i.e. from where it can be accessed in your application. In Java, there 4 type of the access specifiers.
 - **public**: accessible in all class in your application.
 - **protected**: accessible within the class in which it is defined and in its **subclass(es)**
 - **private**: accessible only within the class in which it is defined.
 - **default (declared/defined without using any modifier)** : accessible within same class and package within which its class is defined.
- **The return type** : The data type of the value returned by the the method or void if does not return a value.
- **Method Name** : the rules for field names apply to method names as well, but the convention is a little different.
- **Parameter list** : Comma separated list of the input parameters are defined, preceded with their data type, within the enclosed parenthesis. If there are no parameters, you must use empty parentheses ().
- **Exception list** : The exceptions you expect by the method can throw, you can specify these exception(s).
- **Method body** : it is enclosed between braces. The code you need to be executed to perform your intended operations.

Example methods

```
class Add
{
public static void main(String[] arg)
{
int a,b,c;
Scanner sc=new Scanner(System.in);
System.out.println("Enter first number");
a=sc.nextInt();
System.out.println("Enter second number");
b=sc.nextInt();
c=addition(a,b);
System.out.println(" Addition of two numbers is : "+c);
}
static int addition(int x,int y)
{
    return x+y;
}
}
```

1.8 Inheritance in java

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

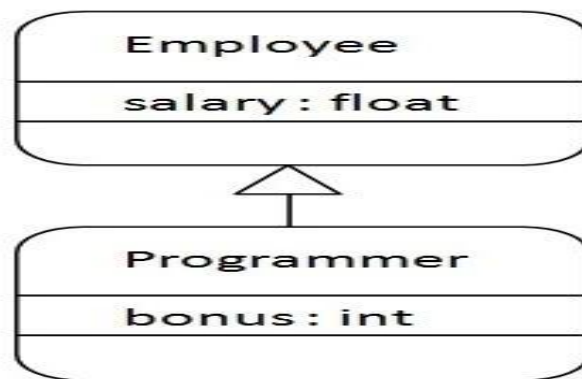
The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.

Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.

Terms used in Inheritance

- **Class:** A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

Java Inheritance Example

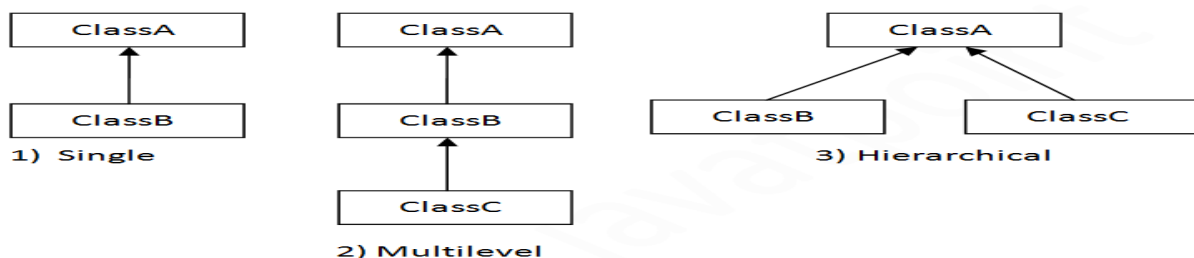


```
class Employee{
    float salary=40000;
}
class Programmer extends Employee{
    int bonus=10000;
    public static void main(String args[]){
        Programmer p=new Programmer();
        System.out.println("Programmer salary is:"+p.salary);
        System.out.println("Bonus of Programmer is:"+p.bonus);
    }
}
```

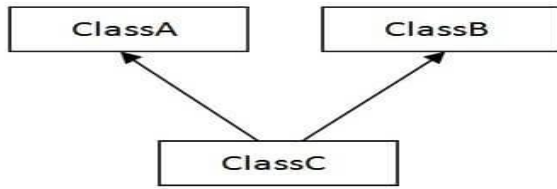
Types of inheritance in java

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.

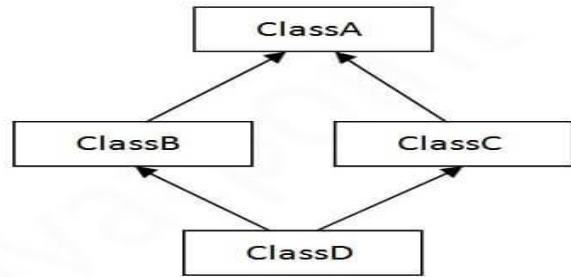
In java programming, multiple and hybrid inheritance is supported through interface only. We will learn about interfaces later.



Note: Multiple inheritance and hybrid is not supported in Java through class.



4) Multiple



5) Hybrid

Example single inheritance

```
class Animal{  
void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}  
class TestInheritance{  
public static void main(String args[]){  
Dog d=new Dog();  
d.bark();  
d.eat();  
}}}
```

1.9 Packages in java

A **java package** is a group of similar types of classes, interfaces and sub-packages.

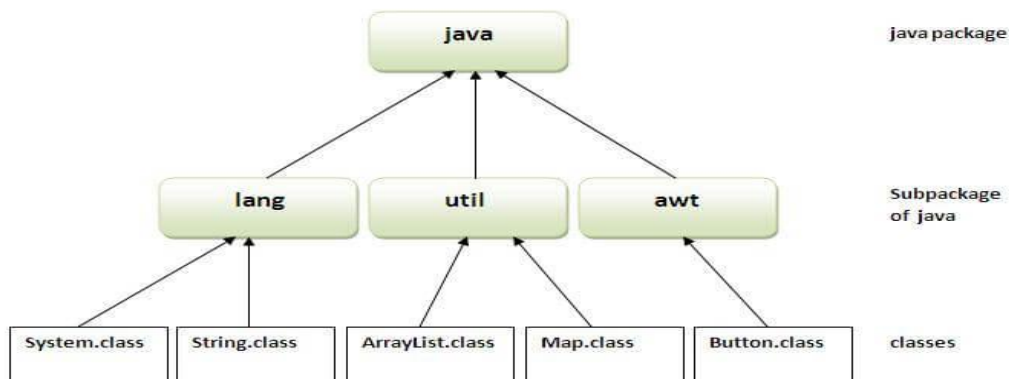
Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Here, we will have the detailed learning of creating and using user-defined packages.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.



Program-1

```
package arithmetic;
public class MyMath
{
    public int add(int x, int y)
    {
        System.out.println ("t" + (x + y));
    }
    public int sub(int x, int y)
    {
        System.out.println ("t" + (x - y));
    }
    public int mul(int x, int y)
    {
        System.out.println ("t" + (x * y));
    }
    public double div(int x, int y)
    {
        System.out.println ("t" + (x / y));
    }
    public int mod(int x, int y)
    {
        System.out.println ("t" + (x % y));
    }
}
```

Compile `javac -d . MyMath.java`

Program-2

```
import arithmetic.*;
class Test
{
    public static void main(String as[])
    {
        MyMath m=new MyMath();
        m.add(13.42);
        m.sub(16,5);
        m.mul(8,4);
        m.div(24,9);
        m.mod(19,6);
    }
}
```

Compile and execute above program normally

1.10 Interfaces in java

Like a class, an interface can have methods and variables, but the methods declared in interface are by default abstract (only method signature, no body).

- Interfaces specify what a class must do and not how. It is the blueprint of the class.
- An Interface is about capabilities like a Player may be an interface and any class implementing Player must be able to (or must implement) move(). So it specifies a set of methods that the class has to implement.
- If a class implements an interface and does not provide method bodies for all functions specified in the interface, then class must be declared abstract.
- A Java library example is, Comparator Interface. If a class implements this interface, then it can be used to sort a collection.

```
// An example to show that interfaces
interface in1
{
    final int a = 10;
    default void display()
    {
        System.out.println("hello");
    }
}
```

```
// A class that implements interface.
class testClass implements in1
{
    // Driver Code
    public static void main (String[] args)
    {
        testClass t = new testClass();
        t.display();    }    }
```

1.11 Inner classes in java

Java inner class is defined inside the body of another class. Java inner class can be declared private, public, protected, or with default access whereas an outer class can have only public or default access.

Java Nested classes are divided into two types.

a. static nested class

If the nested class is static, then it's called static nested class. Static nested classes can access only static members of the outer class. Static nested class is same as any other top-level class and is nested for only packaging convenience.

b. java inner class

Any non-static nested class is known as inner class in java. Java inner class is associated with the object of the class and they can access all the variables and methods of the outer class. Since inner classes are associated with instance, we can't have any static variables in them.

```
class Outer {  
    // Simple nested inner class  
    class Inner {  
        public void show() {  
            System.out.println("In a nested class method");  
        }  
    }  
}  
class Main {  
    public static void main(String[] args) {  
        Outer.Inner in = new Outer().new Inner();  
        in.show();  
    }  
}
```

Output:

In a nested class method

END OF UNIT-1